

Agentic AI Engineering

A Complete Guide

8 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/ai-agentic-track/>

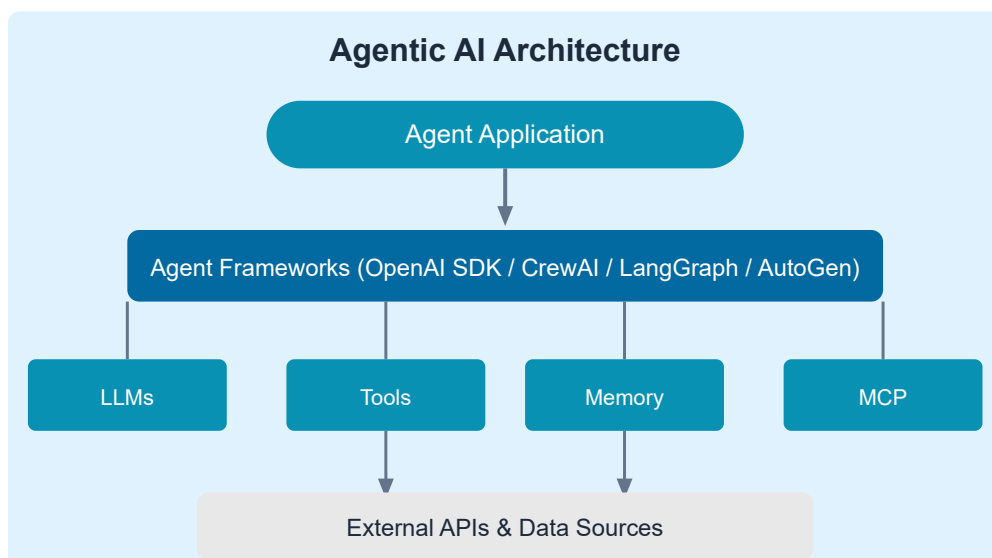
Foundation Week

Introduction & Setup

Start with a smart home demo to understand what agents are, then explore the frameworks landscape, career paths, and IDE setup for Windows and Mac.

Core Design Patterns

Pattern	Description
Agent Autonomy	Agents make decisions independently based on goals
Sequential Workflow	Step-by-step chain of LLM calls
Routing	Direct input to specialized sub-agents
Parallelization	Fan-out to multiple models simultaneously



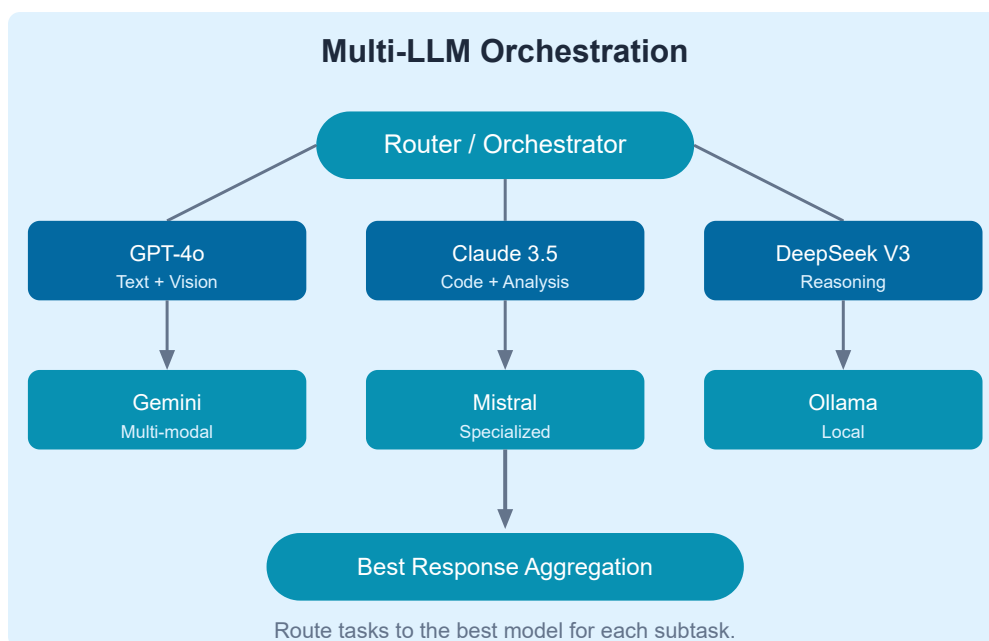
Multi-LLM Orchestration

Compare GPT-4o, Claude, Gemini, DeepSeek. Each has strengths — route tasks to the right model.

```
# Multi-LLM router example
async def route_task(task_type, prompt):
    if task_type == "vision":
        return await call_gpt4o(prompt)
    elif task_type == "code":
        return await call_claude(prompt)
    elif task_type == "reasoning":
        return await call_deepseek(prompt)
    else:
        return await call_gemini(prompt)
```

Frameworks & UI Tools

- **OpenAI Agents SDK** — Lightweight, Python-native agent framework
- **CrewAI** — Role-based multi-agent collaboration
- **LangGraph** — Graph-based stateful agent orchestration
- **AutoGen** — Multi-agent conversation framework from Microsoft
- **Gradio** — Rapid UI prototyping for agent demos



Deployment & Wrap-up

Learn LLM function calling, tool handling, AI assistant patterns, and deploying agents to production. End with a career chatbot project.

 **Exercise:** Set up your development environment (Python 3.11+, VS Code, Git). Build a simple multi-step LLM workflow that takes a user query, classifies it (technical, billing, general), and routes to a specialized prompt. Use at least two different LLM providers.

[← Back to Tutorials](#)

OpenAI Agents SDK

SDK Fundamentals

The OpenAI Agents SDK provides a lightweight, Python-native way to build agents. Core classes: `Agent`, `Runner`, and `Trace`.

```
pip install openai-agents python-dotenv
```

Basic Agent

```
from agents import Agent, Runner

agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant.",
    model="gpt-4o"
)

result = Runner.run_sync(agent, "What is the capital of France?")
print(result.final_output)
```

Sales Automation & Handoffs

Create specialized sales agents and use handoffs to route between them.

```

from agents import Agent, Runner, handoff

triage_agent = Agent(
    name="Triage Agent",
    instructions="Route to the right sales agent.",
    handoffs=[
        handoff(Agent(name="Product Agent", instructions="...")),
        handoff(Agent(name="Pricing Agent", instructions="...")),
    ]
)

```

Agents as Tools

One agent can call another agent as a tool within its own workflow.

```

from agents import Agent, function_tool

@function_tool
def search_knowledge_base(query: str) -> str:
    """Search the knowledge base for information."""
    return f"Results for: {query}"

agent = Agent(
    name="Support Agent",
    tools=[search_knowledge_base]
)

```

Safety & Guardrails

Add input and output guardrails to prevent misuse.

```

from agents import Guardrail

class PIIGuardrail(Guardrail):
    async def check_output(self, text: str):
        # Check for PII patterns
        if "@" in text and ".com" in text:
            return False, "Output contains email address"
        return True, None

```

Deep Research Workflows

Build agents that search the web, plan research, and compile reports.

Visualization & Deployment

Use Gradio to add a chat UI, then deploy to HuggingFace Spaces.

```

import gradio as gr

def chat(message, history):
    result = Runner.run_sync(agent, message)
    return result.final_output

gr.ChatInterface(chat, title="AI Agent").launch()

```

 **Exercise:** Build a sales outreach automation system with three agents: a Prospect Research Agent, a Personalized Outreach Agent, and a Follow-up Agent. Use handoffs to pass context between them. Add input guardrails to block PII. Deploy the final system with a Gradio chat interface.

[← Back to Tutorials](#)

CrewAI Framework

Multi-Agent Collaboration

CrewAI enables role-based agent teams. Each agent has a role, goal, and backstory — forming a crew that works together on tasks.

```
pip install crewai crewai-tools
```

Creating a Crew

```
from crewai import Agent, Task, Crew

researcher = Agent(
    role="Senior Researcher",
    goal="Find accurate information",
    backstory="Expert researcher with 20 years experience",
    tools=[],
    verbose=True
)

writer = Agent(
    role="Content Writer",
    goal="Write compelling content based on research",
    backstory="Award-winning technology writer",
)

research_task = Task(
    description="Research the latest AI agent frameworks",
    agent=researcher,
    expected_output="A detailed report on AI agent frameworks"
)

write_task = Task(
    description="Write a blog post from the research",
    agent=writer,
    expected_output="A 500-word blog post"
)

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    verbose=True
)

result = crew.kickoff()
```

Financial Research Systems

Build multi-agent financial research teams that gather data, analyze trends, and generate reports.

Custom Tools & Structured Outputs

```
from crewai_tools import tool
from pydantic import BaseModel

class StockAnalysis(BaseModel):
    ticker: str
    current_price: float
    recommendation: str

@tool("Stock Price Tool")
def get_stock_price(ticker: str) -> float:
    """Get the current stock price for a ticker."""
    import yfinance as yf
    stock = yf.Ticker(ticker)
    return stock.info.get("currentPrice", 0)
```

Memory & Coding Agents

CrewAI supports short-term, long-term, and entity memory. Use SQL or vector storage for persistent memory across runs.

Advanced Trading Applications

Combine multiple agents (data collector, analyst, risk assessor, trader) into a collaborative trading framework.

 **Exercise:** Build a financial research crew with three agents: a Data Collector (fetches stock prices and news), an Analyst (evaluates performance and trends), and a Report Writer (generates structured Markdown reports). Use Pydantic for structured outputs. Run the crew for 3 different tickers.

LangGraph Architecture

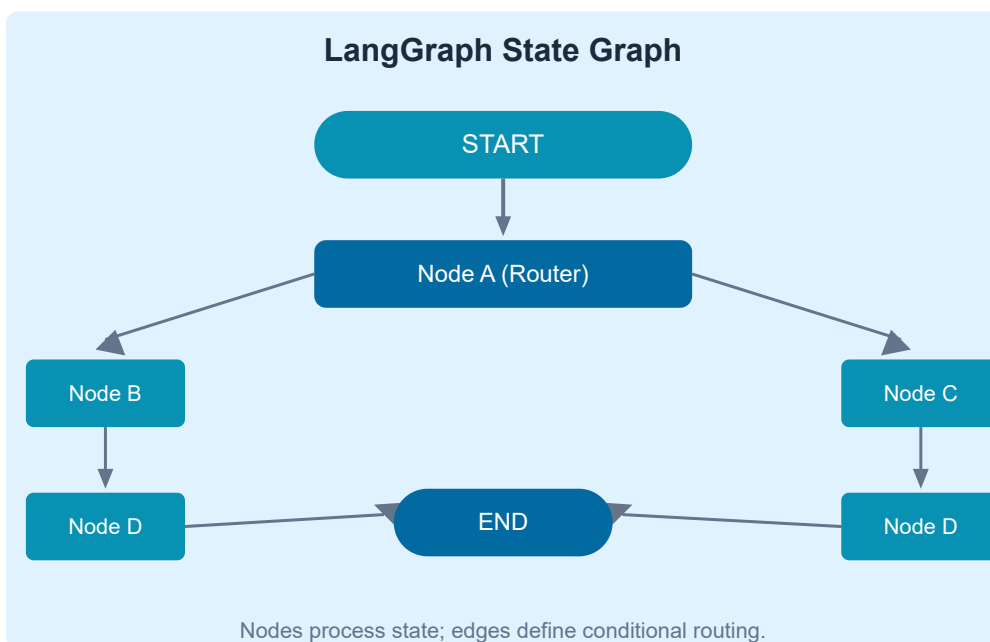
Graph-Based Design

LangGraph models agent workflows as state machines. Nodes process state; edges define conditional transitions. This enables loops, branching, and complex orchestration.

```
pip install langgraph langchain-openai
```

Core Components

- **State** — Shared dictionary passed between nodes
- **Node** — A function that processes state
- **Edge** — Defines the flow between nodes
- **Conditional Edge** — Route based on state content



State Management

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class AgentState(TypedDict):
    messages: List[str]
    next_agent: str

def router_node(state: AgentState):
    # Classify and route
    if "billing" in state["messages"][-1].lower():
        return {"next_agent": "billing"}
    return {"next_agent": "support"}

def billing_node(state: AgentState):
    return {"messages": [f"Billing: Processing..."]}

graph = StateGraph(AgentState)
graph.add_node("router", router_node)
graph.add_node("billing", billing_node)
graph.add_conditional_edges("router", lambda s: s["next_agent"])
graph.set_entry_point("router")
app = graph.compile()
```

Advanced Features

- **Checkpointing** — Save and restore state for fault tolerance
- **Persistent Memory** — Store conversation history across sessions
- **Super Steps** — Batch multiple LLM calls in one node

Browser Automation

Integrate Playwright agents as LangGraph nodes for web browsing, form filling, and data extraction.

Feedback Loops & Evaluators

Add evaluator nodes that critique outputs and trigger re-processing if quality is insufficient.

```
def evaluator_node(state: AgentState):  
    output = state["messages"][-1]  
    if len(output) < 50:  
        return {"needs_revision": True}  
    return {"needs_revision": False}
```

 **Exercise:** Build a LangGraph chatbot with three nodes: (1) a classifier node that determines intent, (2) a response node that generates answers, and (3) an evaluator node that checks response quality and triggers revision if needed. Use checkpointing to persist state across interruptions.

[← Back to Tutorials](#)

Microsoft AutoGen

Framework Overview

AutoGen is Microsoft's multi-agent conversation framework. Agents communicate via messages, support hierarchical conversation patterns, and can run distributed across machines.

```
pip install pyautogen
```

Agent Chat Tutorial

```
from autogen import AssistantAgent, UserProxyAgent

assistant = AssistantAgent(
    name="Assistant",
    llm_config={"config_list": [{"model": "gpt-4o", "api_key": "..."}]}
)

user = UserProxyAgent(
    name="User",
    human_input_mode="NEVER",
    code_execution_config=False
)

user.initiate_chat(
    assistant,
    message="Explain the concept of agentic AI in simple terms."
)
```

Advanced Interaction Patterns

Pattern	Description
Two-Agent Chat	Basic conversation between assistant and user
Group Chat	Multiple agents discuss with a manager
Sequential Chat	Chain of agent conversations
Nested Chat	Chats within chats for complex workflows

Multimodal Features

AutoGen agents can process images, execute code, and scrape web content.

```
# Multimodal agent
assistant = AssistantAgent(
    name="MultimodalAssistant",
    llm_config={"config_list": [{"model": "gpt-4o"}]}
)

# Agent can analyze images sent as messages
```

AutoGen Core (Distributed)

AutoGen Core enables distributed agent communication via gRPC. Agents can run on different machines and communicate asynchronously.

Distributed Runtimes

```
# Agent running on a remote worker
from autogen.runtime import GrpcWorkerAgent

worker = GrpcWorkerAgent(
    name="Worker1",
    address="localhost:50051"
)

# Orchestrator sends tasks to workers
orchestrator.send_message(worker, "Process this data")
```

Self-Deploying Agents

Build agents that can autonomously deploy code, spin up new agents, and manage their own lifecycle.

 **Exercise:** Build a group chat with three AutoGen agents: a Coordinator, a Researcher, and a Writer. The Coordinator manages the conversation, the Researcher fetches information, and the Writer generates the final output. Implement a sequential chat pattern where research results are passed to the writer.

[← Back to Tutorials](#)

MCP & Capstone

Introduction to MCP

The Model Context Protocol (MCP) standardizes how agents discover and call tools. MCP has three roles:

- **Host** — The application that runs the agent (e.g., Claude Desktop, agent framework)
- **Client** — Connects to MCP servers on behalf of the host
- **Server** — Exposes tools, resources, and prompts

Multi-Server Agent

```
from mcp import ClientSession
from contextlib import AsyncExitStack

class MCPServer:
    async def connect(self, server_command: str):
        # Connect to an MCP server process
        self.session = await ClientSession.create(server_command)

    async def list_tools(self):
        return await self.session.list_tools()

    async def call_tool(self, name: str, args: dict):
        return await self.session.call_tool(name, args)
```

Building Custom MCP Servers

```
from mcp.server import Server

server = Server("market-data-server")

@server.list_tools()
async def list_tools():
    return [
        Tool(name="get_stock_price",
              description="Get current stock price",
              inputSchema={"type": "object", "properties": {"ticker": {"type":
    ]

@server.call_tool()
async def call_tool(name, arguments):
    if name == "get_stock_price":
        import yfinance as yf
        stock = yf.Ticker(arguments["ticker"])
        return {"price": stock.info.get("currentPrice")}
```

Memory & External APIs

Build MCP servers that connect to Brave Search API, Polygon.io (market data), and other external services.

Capstone: Trading Floor

The final capstone combines everything into a trading floor simulation:

- **Market Data MCP Server** — Fetches real-time stock data
- **News Analysis MCP Server** — Analyzes market news sentiment
- **Portfolio Manager Agent** — Makes trading decisions
- **Risk Assessment Agent** — Evaluates portfolio risk
- **UI Dashboard** — Gradio-based trading interface

Course Conclusion

Learn how to select the right framework for your use case, key configuration settings, and lessons learned from production agent deployments.

 **Exercise:** Build a custom MCP server that provides a portfolio management tool (check balance, buy/sell stocks, view history). Create an agent that uses this MCP server along with a market data MCP server to provide a complete trading assistant. Deploy with a Gradio frontend.

7. Practice & Resources

Quick Reference Guide

Task	OpenAI SDK	CrewAI	LangGraph	AutoGen
Create agent	<code>Agent(name, instructions)</code>	<code>Agent(role, goal)</code>	<code>add_node(name, fn)</code>	<code>AssistantAgent(name)</code>
Add tool	<code>@function_tool</code>	<code>@tool</code>	Include in state	<code>register_function</code>
Multi-agent	<code>handoff()</code>	<code>Crew(agents)</code>	Multiple nodes	<code>GroupChat</code>
Memory	Custom impl	Built-in	State + checkpoint	Conversation history
Structured output	<code>output_schema</code>	Pydantic	TypedDict state	Custom models

Framework Selection Guide

Use Case	Best Framework
Simple assistant with tools	OpenAI Agents SDK
Role-based team collaboration	CrewAI
Complex stateful workflows	LangGraph
Distributed multi-agent systems	AutoGen
Custom tool ecosystems	MCP + any framework

Useful Resources

- [OpenAI Agents SDK Docs](#)
- [CrewAI Documentation](#)
- [LangGraph Documentation](#)
- [AutoGen Documentation](#)
- [MCP Specification](#)

6-Week Study Plan

Week	Topic	Activities
1	Foundation Week	Setup, patterns, multi-LLM, frameworks overview
2	OpenAI Agents SDK	SDK fundamentals, sales agents, guardrails, deployment
3	CrewAI	Multi-agent crews, financial research, custom tools
4	LangGraph	State graphs, chatbots, browser automation, checkpointing
5	AutoGen	Group chat, multimodal, distributed runtimes
6	MCP + Capstone	MCP servers, trading floor capstone, framework selection

Certificate Path

- **AI-900:** Azure AI Fundamentals
- **AI-102:** Azure AI Engineer Associate
- **OpenAI Developer Certification**

 **Final Capstone:** Build an end-to-end multi-agent trading system using at least two different frameworks. Include: (1) a market data retrieval agent, (2) a news sentiment analysis agent, (3) a portfolio management agent, (4) MCP servers for data access, (5) a Gradio UI, and (6) deployment to HuggingFace Spaces.

8. References

Glossary

Term	Definition
Agent	Autonomous AI entity that uses LLMs, tools, and memory to accomplish goals
Tool	A function an agent can call to perform actions (APIs, code, search)
Handoff	Transferring conversation control from one agent to another
Orchestrator	Manages agent coordination, routing, and workflow sequencing
MCP	Model Context Protocol — standardized interface for agent tools
RAG	Retrieval-Augmented Generation — grounding LLM responses in data
Structured Output	LLM returns typed, schema-validated data instead of free text
Guardrail	Safety policy applied to agent inputs and outputs

Cheatsheet

```
# Installation Cheatsheet

# OpenAI Agents SDK
pip install openai-agents

# CrewAI
pip install crewai crewai-tools

# LangGraph
pip install langgraph langchain-openai

# AutoGen
pip install pyautogen

# MCP
pip install mcp

# Gradio (UI)
pip install gradio

# Common imports
from agents import Agent, Runner, handoff
from crewai import Agent, Task, Crew, Process
from langgraph.graph import StateGraph, END
from autogen import AssistantAgent, UserProxyAgent
```

API Reference

Framework	Core Classes / Functions
OpenAI SDK	<code>Agent</code> , <code>Runner</code> , <code>Trace</code> , <code>handoff</code> , <code>function_tool</code> , <code>Guardrail</code>
CrewAI	<code>Agent</code> , <code>Task</code> , <code>Crew</code> , <code>Process</code> , <code>@tool</code> , <code>knowledge</code>
LangGraph	<code>StateGraph</code> , <code>add_node</code> , <code>add_edge</code> , <code>compile</code> , <code>Checkpointter</code>
AutoGen	<code>AssistantAgent</code> , <code>UserProxyAgent</code> , <code>GroupChat</code> , <code>GroupChatManager</code>
MCP	<code>Server</code> , <code>ClientSession</code> , <code>Tool</code> , <code>list_tools</code> , <code>call_tool</code>

Key Lessons Learned

- Start simple — add complexity only when needed
- Use structured outputs for reliable data exchange
- Always add guardrails for production agents
- Memory is critical for multi-turn conversations
- MCP makes tools reusable across frameworks
- Monitor and trace agent decisions in production

 **Congratulations!** You've completed the AI Engineer Agentic Track. You now have hands-on experience with all major agent frameworks and can build sophisticated multi-agent systems for real-world applications.